



MONASH University

Information Technology

# FIT3143 Parallel Computing Semester 2 2026

**Applied #1 (Week 5)**

**Parallel Computing In Distributed Memory – Message  
Passing Library**

# Overview

1. **Message Passing Interface (MPI) (60 minutes)**
2. **Practice questions (30 minutes)**
3. **Week 6 - Applied Tasks overview (15 minutes)**

# Message Passing Interface (MPI)

# What is MPI

## M P I = Message Passing Interface

MPI is a specification for the developers and users of message passing libraries. By itself, *it is NOT a library* - but rather the *specification* of what such a library should be.

Simply stated, the goal of the Message Passing Interface is to provide a widely used standard for writing message passing programs. The interface attempts to be

- practical
- portable
- efficient
- flexible

# Reasons for Using MPI

- **Standardization** - Supported on all major platforms and also many specialised HPC systems. (Practically has replaced all previous message passing libraries)
- **Portability** - Minimum (to none) need to modify your source code when you port your application to a different platform compliant with MPI (standard).
- **Performance Opportunities** - Vendor implementations can exploit native hardware features to optimize performance.
- **Functionality** - About 128 functions/routines in MPI 1.3, finalised in 2008. By 2026, MPI has reached 5.0.
- **Availability** - A variety of implementations on the standard are available (vendor and public domain implementations).

(For Applied/Lab assessment: [OpenMPI library implementations](#))

# Programming Model

- MPI supports almost any popular distributed memory parallel programming model.
- MPI can also be used to implement/support many shared memory models.
- Hardware platforms:
  - Distributed Memory: Primarily targeted for distributed memory systems.
  - Shared Memory: As shared memory systems became more popular (SMP / NUMA architectures), MPI implementations for these platforms appeared.
  - Hybrid: MPI supports most common hybrid architecture (massively parallel machines, SMP clusters, workstation clusters and heterogeneous networks, etc).

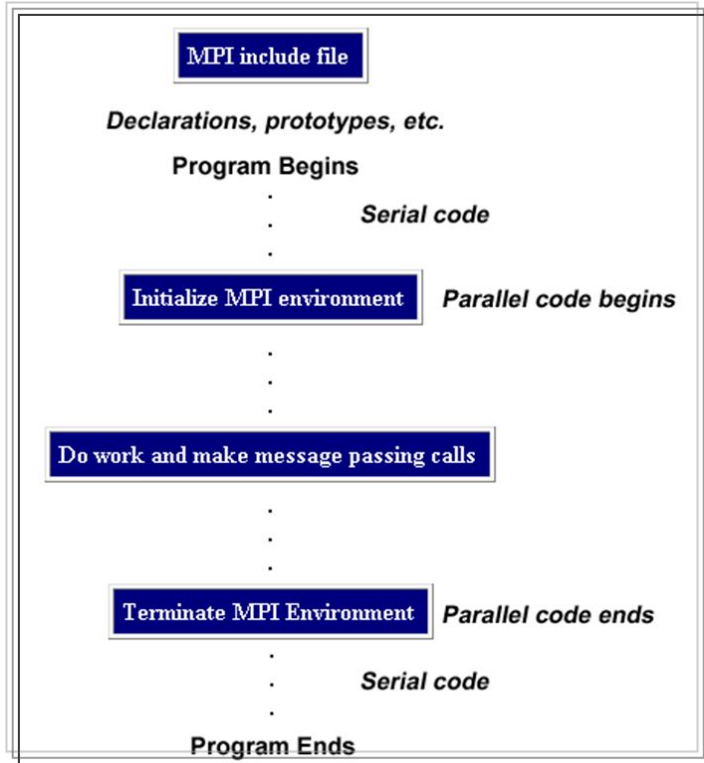
# Programming Model

- All parallelism is explicit!
- *The programmer is responsible* for correctly identifying parallelism and implementing parallel algorithms using MPI constructs.
- Early version (MPI-1):
  - Number of tasks dedicated to run a parallel program is static, i.e., new tasks can not be dynamically spawned during run time.

# Getting Started

- MPI is native to ANSI C
  - C++ and Java bindings are available
    - MPI C++ classes – [www.mcs.anl.gov](http://www.mcs.anl.gov)
    - mpiJava API – [www.hpjava.org](http://www.hpjava.org)
- Make sure you check your *MPI versions and implementation libraries!*
- MPI - C
  - Header File:
    - Required for all programs/routines which make MPI library calls.  
`#include "mpi.h"`  
Or  
`#include <mpi.h>`

# General MPI Program Structure



## Format of MPI Calls:

- Format: `rc = MPI_Xxxxx(parameter, ... )`
- Example: `rc = MPI_Bsend(&buf,count,type,dest,tag,comm)`
- Error code: `rc` value is set to `MPI_SUCCESS` if successful

# MPI's “Hello World”

## 1. **Create** Source Code File: hello.c

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int numprocs, rank, namelen;
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Get_processor_name(processor_name, &namelen);

    printf("Process %d on %s out of %d\n", rank, processor_name, numprocs);

    MPI_Finalize();
}
```

## 2. **Compile:** mpicc hello.c -o hello-mp

## 3. **Execute:** mpirun -np 2 hello-mp

# Communicators and Groups

- MPI uses objects called communicators and groups to define which collection of processes may communicate with each other.
- Most MPI routines require you to specify a communicator (in the argument list).
- **MPI\_COMM\_WORLD** is the predefined communicator that includes all of your MPI processes.  
(We will cover different communicators and groups later)

# MPI's "Hello World"

## 1. Create Source Code File: hello.c

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int numprocs, rank, namelen;
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Get_processor_name(processor_name, &namelen);

    printf("Process %d on %s out of %d\n", rank, processor_name, numprocs);

    MPI_Finalize();
}
```

## 2. Compile: mpicc hello.c -o hello-mp

## 3. Execute: mpirun -np 2 hello-mp

# Communicators and Groups (Rank)

- Every process has its own *unique, integer identifier* (called rank) associated with every communicator.
- Ranks are contiguous and begin at zero. Ranks across communicators may not be unique, i.e., a process may have many different identifiers.
- Rank sometimes will also be called "task ID".
- Used by programmers to specify the source and destination of messages. Often used to control program execution (if rank == 0 do MPI\_Send() ).

# MPI's "Hello World"

## 1. Create Source Code File: hello.c

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int numprocs, rank, namelen;
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Get_processor_name(processor_name, &namelen);

    printf("Process %d on %s out of %d\n", rank, processor_name, numprocs);

    MPI_Finalize();
}
```

## 2. Compile: mpicc hello.c -o hello-mp

## 3. Execute: mpirun -np 2 hello-mp

# Basic MPI functions

# Environment Management Routines

## **MPI\_Init**

Initializes the MPI execution environment. This function must be called in every MPI program, must be called before any other MPI functions and must be called only once in an MPI program. For C programs, MPI\_Init may be used to pass the command line arguments to all processes, although this is not required by the standard and is implementation dependent.

**MPI\_Init (&argc, &argv)**

## **MPI\_Comm\_size**

Determines the number of processes in the group associated with a communicator. Generally used within the communicator MPI\_COMM\_WORLD to determine the number of processes being used by your application.

**MPI\_Comm\_size (comm, &size)**

# Environment Management Routines

## **MPI\_Comm\_rank**

Determines the rank of the calling process within the communicator. Initially, each process will be assigned a unique integer rank between 0 and number of processors - 1 within the communicator MPI\_COMM\_WORLD. This rank is often referred to as a task ID.

**MPI\_Comm\_rank (comm, &rank)**

## **MPI\_Abort**

Terminates all MPI processes associated with the communicator. In most MPI implementations it terminates ALL processes regardless of the communicator specified.

**MPI\_Abort (comm, errorcode)**

# Environment Management Routines

## **MPI\_Get\_processor\_name**

Returns the processor name. Also returns the length of the name. The buffer for "name" must be at least MPI\_MAX\_PROCESSOR\_NAME characters in size. What is returned into "name" is implementation dependent - may not be the same as the output of the "hostname" or "host" shell commands.

**MPI\_Get\_processor\_name (&name, &resultlength)**

## **MPI\_Initialized**

Indicates whether MPI\_Init has been called - returns flag as either logical true (1) or false(0). MPI requires that MPI\_Init be called once and only once by each process. This may pose a problem for modules that want to use MPI and are prepared to call MPI\_Init if necessary. MPI\_Initialized solves this problem.

**MPI\_Initialized (&flag)**

# Environment Management Routines

## MPI\_Wtime

Returns an elapsed wall clock time in seconds (double precision) on the calling processor.

**MPI\_Wtime ()**

## MPI\_Wtick

Returns the resolution in seconds (double precision) of MPI\_Wtime.

**MPI\_Wtick ()**

## MPI\_Finalize

Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program - no other MPI routines may be called after it.

**MPI\_Finalize ()**

# Environment Management Routines Example

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
    int numtasks, rank, rc;
    rc = MPI_Init(&argc,&argv);
    if (rc != MPI_SUCCESS)
    { printf ("Error starting MPI program. Terminating.\n");
      MPI_Abort(MPI_COMM_WORLD, rc);
    }

    MPI_Comm_size(MPI_COMM_WORLD,&numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    printf ("Number of tasks= %d My rank= %d\n", numtasks,rank);

    /***** do some work *****/
    MPI_Finalize();
}
```

# MPI Point-to-Point Communication

# Point to Point (P2P) Communication

## General Concepts

- **Types of Point-to-Point Operations:**

- Message passing between two different MPI tasks (process). One task is performing a send operation and the other task is performing a matching receive operation.
- Different types of send and receive routines for different purposes:
  - Synchronous
  - Blocking
  - Non-blocking
  - Buffered
  - Routines combining send & receive
- MPI also provides several routines associated with send - receive operations, such as those used to wait for a message's arrival or probe to find out if a message has arrived.

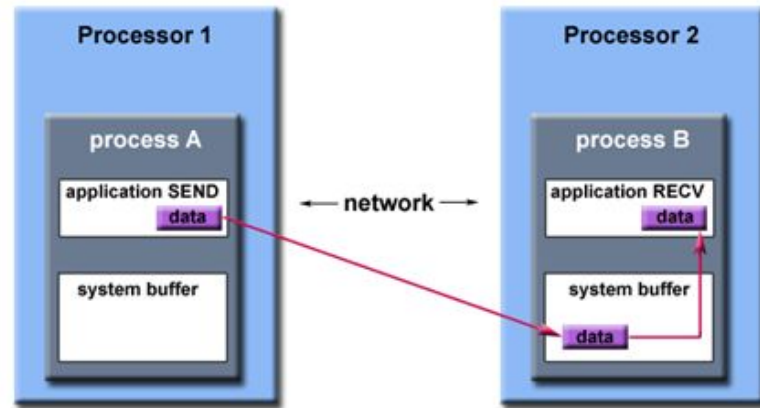
# P2P general concepts

## Buffering:

In a perfect world, every send operation should be perfectly synchronized with its matching receive. In practice, this is rarely the case. The MPI implementation must be able to handle tasks that are out of sync.

Consider the following two cases:

- A send operation occurs 5 seconds before the receive is ready - where the message is stored?
- Multiple sends arrive at the same receiving task which can only accept one send at a time - what happens to the messages that are "backing up"?



Path of a message buffered at the receiving process

The MPI implementation (not the MPI standard) decides what happens to the data in these types of cases. Typically, a system buffer area is reserved to hold data in transit.

# P2P general concepts

**System buffer space** is:

- Managed entirely by the MPI library implementation
- A finite resource that can be easily exhausted
- Often mysterious and not well documented
- May exist on the sending side, the receiving side, or both
- Can be used to improve program performance because it allows send - receive operations to be asynchronous.
- User managed address space (e.g., storing your program variables) is called the **application buffer**.

# P2P general concepts

## Blocking vs. Non-blocking:

- Most of the MPI point-to-point routines can be used in either blocking or non-blocking mode.
- Blocking:
  - A blocking send routine will only "return" after it is safe to modify the application buffer (your send data) for reuse. "Safe" does not imply that the data was actually received. Data may still be sitting in system buffer waiting to be sent.
  - Synchronous blocking send: Block until handshake occurs with the receive task (process).
  - Asynchronous blocking send: Block until data is copied to the system buffer, which is used to hold the data for the "eventual" delivery.
  - Blocking receive: Blocks until after the data has arrived and is ready for use by the program.
- Your parallel design will determine whether you should use blocking/non-block functions. Deadlocks, computation errors, or failing to speed-up the program are common symptoms of using the wrong functions with the wrong parallel design.

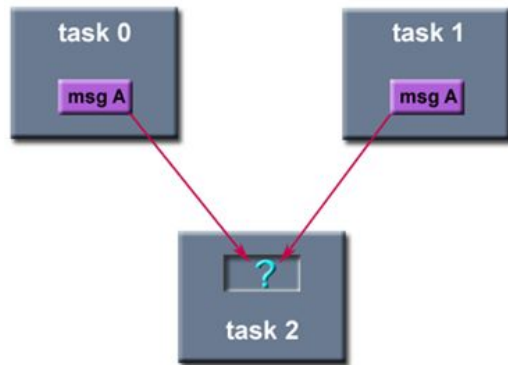
# P2P general concepts

- Non-blocking:
  - Non-blocking send and receive routines will return “almost” immediately.
  - They do not wait for any communication events to complete, such as message copying from user memory to system buffer space or the actual arrival of message.
  - Non-blocking operations simply "request" the MPI library to perform the operation.
  - Corollary: It is unsafe to modify the application buffer (your variable space) until you know the requested operation has been completed. A series of "waiting"/"probing" routines are designed for checking whether operations have been completed.
  - Non-blocking communications are primarily used to overlap computation with communication and exploit possible performance gains

# P2P general concepts

## Order:

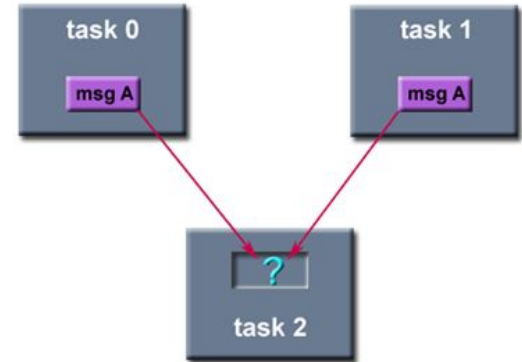
- MPI guarantees that messages are “ordered” and will not overtake each other.
- If a sender sends two messages (Message 1 and Message 2) in succession to the same destination, and both match the same receive, the receive operation will receive Message 1 before Message 2.
- If a receiver posts two receives (Receive 1 and Receive 2), in succession, and both are looking for the same message, Receive 1 will receive the message before Receive 2.
- Order rules do not apply if there are multiple threads participating in the communication operations.



# P2P general concepts

## Fairness:

- MPI does not guarantee fairness
- The programmer has responsibility to prevent operation/process starvation.
- Often depends on how processes are scheduled in the underlying Operating System.



# Point to Point Communication Routines and Arguments

## MPI Message Passing Routine Arguments

Blocking sends

`MPI_Send(buffer, count, type, dest, tag, comm)`

Non-blocking sends

`MPI_Isend(buffer, count, type, dest, tag, comm, request)`

Blocking receive

`MPI_Recv(buffer, count, type, source, tag, comm, status)`

Non-blocking receive

`MPI_Irecv(buffer, count, type, source, tag, comm, request)`

## Buffer

- Program (application) address space that references the data that is to be sent or received.
- In most cases, this is simply the variable name that is to be sent/received. For C programs, this argument is passed by reference (remember to pass variable addresses &var1)

# Point to Point Communication Routines and Arguments

Blocking sends

`MPI_Send(buffer, count, type, dest, tag, comm)`

Non-blocking sends

`MPI_Isend(buffer, count, type, dest, tag, comm, request)`

Blocking receive

`MPI_Recv(buffer, count, type, source, tag, comm, status)`

Non-blocking receive

`MPI_Irecv(buffer, count, type, source, tag, comm, request)`

## Data Count

- Indicates the number of data elements of a particular type to be sent.

## Data Type

- For reasons of portability, MPI pre-defines the allowed data types.

# Point to Point Communication Routines and Arguments

Blocking sends

`MPI_Send(buffer, count, type, dest, tag, comm)`

Non-blocking sends

`MPI_Isend(buffer, count, type, dest, tag, comm, request)`

Blocking receive

`MPI_Recv(buffer, count, type, source, tag, comm, status)`

Non-blocking receive

`MPI_Irecv(buffer, count, type, source, tag, comm, request)`

# MPI Message Passing Routine Arguments

## Destination

- An argument to send routines that indicates the process where a message should be delivered. Specified as the rank of the receiving process.

## Source

- An argument to receive routines that indicates the originating process of the message. Specified as the rank of the sending process. This may be set to the wild card MPI\_ANY\_SOURCE to receive a message from any task.

## Tag

- Arbitrary non-negative integer assigned by the programmer to uniquely identify a message. Send and receive operations should match message tags. For a receive operation, the wild card MPI\_ANY\_TAG can be used to receive any message regardless of its tag.
- The MPI standard guarantees that integers 0-32767 can be used as tags, but most implementations allow a much larger range than this.

# Point to Point Communication Routines and Arguments

Blocking sends

`MPI_Send(buffer, count, type, dest, tag, comm)`

Non-blocking sends

`MPI_Isend(buffer, count, type, dest, tag, comm, request)`

Blocking receive

`MPI_Recv(buffer, count, type, source, tag, comm, status)`

Non-blocking receive

`MPI_Irecv(buffer, count, type, source, tag, comm, request)`

## Communicator

- Indicates the communication context, or set of processes for which the source or destination fields are valid.
- Unless the programmer is explicitly creating new communicators, the predefined communicator `MPI_COMM_WORLD` is commonly used.

# Point to Point Communication Routines and Arguments

Blocking sends

`MPI_Send(buffer, count, type, dest, tag, comm)`

Non-blocking sends

`MPI_Isend(buffer, count, type, dest, tag, comm, request)`

Blocking receive

`MPI_Recv(buffer, count, type, source, tag, comm, status)`

Non-blocking receive

`MPI_Irecv(buffer, count, type, source, tag, comm, request)`

# MPI Message Passing Routine Arguments

## Status

- For a receive operation, indicates the status of the message (e.g., source, tag, and error status).
- In C, this argument is a pointer to a predefined structure `MPI_Status`.

## Request (primarily for non-blocking send and receive operations)

- Since non-blocking operations will immediately return before completion of the corresponding operations, the system/library will issue a unique handle / "request number".
- The programmer can use this handle for future operations (e.g., "waiting" type of routines)
- In C, this argument is a pointer to a predefined structure `MPI_Request`.

# Send and Receive Blocking Variants

# Blocking Message Passing Routines

## MPI\_Send

- Routine **blocks and returns only after the application buffer in the sending task is free for reuse**. Note that this routine may be implemented differently on different systems. The MPI standard permits the use of a system buffer but does not require it. [Some implementations may actually use a synchronous send to implement the basic blocking send.]

**MPI\_Send (&buf,count,datatype,dest,tag,comm)**

## MPI\_Recv

- Receive a message and **block until the requested data is available** in the application buffer in the receiving task.

**MPI\_Recv (&buf,count,datatype,source,tag,comm,&status)**

# Blocking Message Passing Routines

## MPI\_Ssend

- Synchronous blocking send: Send a message and **block until the application buffer in the sending task is free for reuse and the destination process has started to receive the message.**

**MPI\_Ssend (&buf,count,datatype,dest,tag,comm)**

## MPI\_Bsend

- Buffered blocking send: permits the programmer to allocate the required amount of buffer space into which data can be copied until it is delivered. Insulates against the problems associated with insufficient system buffer space. Routine **returns after the data has been copied from application buffer space to the allocated send buffer.** Must be used with the MPI\_Buffer\_attach routine.

**MPI\_Bsend (&buf,count,datatype,dest,tag,comm)**

# Blocking Message Passing Routines

## MPI\_Buffer\_attach

- Used by programmer to allocate/deallocate message buffer space to be used by the MPI\_Bsend routine. The size argument is specified in actual data bytes - not a count of data elements. Only one buffer can be attached to a process at a time. Note that the IBM implementation uses MPI\_BSEND\_OVERHEAD bytes of the allocated buffer for overhead.

**MPI\_Buffer\_attach (&buffer,size)**

**MPI\_Buffer\_detach (&buffer,size)**

## MPI\_Rsend

- Blocking ready send. A send that uses the ready communication mode **may be started only if the matching receive is already posted.**
- On some systems, this allows the removal of **a hand-shake operation that is otherwise required and results in improved performance.**

**MPI\_Rsend (&buf,count,datatype,dest,tag,comm)**

# Blocking Message Passing Routines

## **MPI\_Sendrecv**

- Send a message and post a receive before blocking.
- Will block until the sending application buffer is free for reuse and until the receiving application buffer contains the received message.

**MPI\_Sendrecv (&sendbuf,sendcount,sendtype,dest,sendtag,  
&recvbuf,recvcount,recvtype,source,recvtag, comm,&status)**

# Blocking Message Passing Routines

**MPI\_Wait**

**MPI\_Waitany**

**MPI\_Waitall**

**MPI\_Waitsome**

- **MPI\_Wait** blocks until a specified non-blocking send or receive operation has completed. For multiple non-blocking operations, the programmer can specify any, all or some completions.

**MPI\_Wait (&request,&status)**

**MPI\_Waitany (count,&array\_of\_requests,&index,&status)**

**MPI\_Waitall (count,&array\_of\_requests,&array\_of\_statuses)**

**MPI\_Waitsome (incount,&array\_of\_requests,&outcount,  
&array\_of\_offsets, &array\_of\_statuses)**

# Blocking Message Passing Routines

## MPI\_Probe

- Performs a **blocking test for a message**.
- The "wildcards" MPI\_ANY\_SOURCE and MPI\_ANY\_TAG may be used to test for a message from any source or with any tag.
- For the C routine, the actual source and tag will be returned in the status structure as status.

**MPI\_Probe (source,tag,comm,&status)**

# Example: Blocking Message Passing Routines

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
    int numtasks, rank, dest, source, rc, count, tag=1;
    char inmsg, outmsg='x';
    MPI_Status Stat;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0)
    {
        dest = 1; source = 1;
        rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag,
                     MPI_COMM_WORLD);
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag,
                     MPI_COMM_WORLD, &Stat);
    }
```

```
    else if (rank == 1)
    {
        dest = 0; source = 0;
        rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag,
                     MPI_COMM_WORLD, &Stat);
        rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag,
                     MPI_COMM_WORLD);
    }
    rc = MPI_Get_count(&Stat, MPI_CHAR, &count);
    printf("Task %d: Received %d char(s) from task %d with\n",
           rank, count, Stat.MPI_SOURCE, Stat.MPI_TAG);
    MPI_Finalize();
}
```

What would happen if MPI\_Recv is posted before MPI\_Send?

# Send and Receive Non-blocking Variants

# Non-Blocking Message Passing Routines

## **MPI\_Isend**

Processing **continues immediately** without waiting for the message to be copied out from the application buffer.

The program should not modify the application buffer until subsequent calls to MPI\_Wait or MPI\_Test indicate that the non-blocking send has completed.

**MPI\_Isend (&buf,count,datatype,dest,tag,comm,&request)**

## **MPI\_Irecv**

Processing **continues immediately** without actually waiting for the message to be received and copied into the the application buffer.

The program must use calls to MPI\_Wait or MPI\_Test to determine when the non-blocking receive operation completes and the requested message is available in the application buffer.

**MPI\_Irecv (&buf,count,datatype,source,tag,comm,&request)**

# Non-Blocking Message Passing Routines

## **MPI\_Issend**

Non-blocking synchronous send.

Similar to MPI\_Isend(), except MPI\_Wait() or MPI\_Test() indicates when the destination process has received the message.

**MPI\_Issend (&buf,count,datatype,dest,tag,comm,&request)**

## **MPI\_IbSEND**

Non-blocking buffered send.

Similar to MPI\_Bsend() except MPI\_Wait() or MPI\_Test() indicates when the destination process has received the message. Must be used with the MPI\_Buffer\_attach routine.

**MPI\_IbSEND (&buf,count,datatype,dest,tag,comm,&request)**

# Non-Blocking Message Passing Routines

## **MPI\_Irsend**

Non-blocking ready send.

Similar to MPI\_Rsend() except MPI\_Wait() or MPI\_Test() indicates when the destination process has received the message.

**MPI\_Irsend (&buf,count,datatype,dest,tag,comm,&request)**

# Non-Blocking Message Passing Routines

**MPI\_Test**

**MPI\_Testany**

**MPI\_Testall**

**MPI\_Testsome**

- MPI\_Test checks the status of a specified non-blocking send or receive operation.
- The "flag" parameter is returned logical true (1) if the operation has completed, and logical false (0) if not.
- For multiple non-blocking operations, the programmer can specify any, all or some completions.

**MPI\_Test (&request,&flag,&status)**

**MPI\_Testany (count,&array\_of\_requests,&index,&flag,&status)**

**MPI\_Testall (count,&array\_of\_requests,&flag,&array\_of\_statuses)**

**MPI\_Testsome (incount,&array\_of\_requests,&outcount, &array\_of\_indices,  
&array\_of\_statuses)**

# Non-Blocking Message Passing Routines

## **MPI\_Iprobe**

- Performs a non-blocking test for a message.
- The "wildcards" MPI\_ANY\_SOURCE and MPI\_ANY\_TAG may be used to test for a message from any source or with any tag.
- The integer "flag" parameter is returned logical true (1) if a message has arrived, and logical false (0) if not.

**MPI\_Iprobe (source,tag,comm,&flag,&status)**

# Example: Non-Blocking Message Passing Routines

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
    int numtasks, rank, next, prev, buf[2],
        tag1=1, tag2=2;
    MPI_Request reqs[4];
    MPI_Status stats[4];
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    prev = rank-1;
    next = rank+1;

    if (rank == 0) prev = numtasks - 1;
    if (rank == (numtasks - 1)) next = 0;
    MPI_Irecv(&buf[0], 1, MPI_INT, prev, tag1,
        MPI_COMM_WORLD, &reqs[0]);
    MPI_Irecv(&buf[1], 1, MPI_INT, next, tag2,
        MPI_COMM_WORLD, &reqs[1]);
    MPI_Isend(&rank, 1, MPI_INT, prev, tag2,
        MPI_COMM_WORLD, &reqs[2]);
    MPI_Isend(&rank, 1, MPI_INT, next, tag1,
        MPI_COMM_WORLD, &reqs[3]);
    { /* do some work */ }
    MPI_Waitall(4, reqs, stats);
    printf("I am proc %d buf[0]=%d\n", rank, buf[0], buf[1]);
    MPI_Finalize();
}
```

# Collective Communication

# Collective Communication

## All or None:

- Collective communication involve **all processes in the scope of a communicator**.
- Programmer's responsibility to ensure all processes within a communicator are correctly participating in any collective operations.

## General types of collective operations:

- Synchronization: processes wait until all members of the group have reached the synchronization point.
- Data Movement: broadcast, scatter/gather, all to all.
- Collective Computation (e.g., reductions): one member of the group collects data from the other members and performs an operation (e.g., min, max, add, multiply) on that data.

# Collective Communication

## Programming Considerations and Restrictions:

- Collective operations are **blocking**.
- Collective communication routines do not have message tag arguments.
- Putting in wrong communicator may result in erroneous computation or deadlock.

# Collective Communication Routines

## MPI\_Barrier

- Creates a barrier synchronization (within a group/subgroup of processes).
- Each task (process) when reaching the MPI\_Barrier call, blocks until all tasks in the group reach the same MPI\_Barrier call.

**MPI\_Barrier (comm)**

## MPI\_Bcast

- Broadcasts (sends) a message from the “root” process to all other processes in the group.

**MPI\_Bcast (&buffer,count,datatype,root,comm)**

# Collective Communication Routines

## MPI\_Scatter

- Distributes distinct messages from a single source task to each task in the group.
- E.g., Distributing different elements of a data array to different processes.

**MPI\_Scatter (&sendbuf,sendcnt,sendtype,&recvbuf,  
..... recvcnt,recvtype,root,comm)**

## MPI\_Gather

- Gathers distinct messages from each task in the group to a single destination task.
- This routine is the reverse operation of MPI\_Scatter.

**MPI\_Gather (&sendbuf,sendcnt,sendtype,&recvbuf,  
..... recvcount,recvtype,root,comm)**

# Collective Communication Routines

## **MPI\_Allgather**

- Concatenation of data to all tasks in a group.
- Each task in the group, in effect, performs a one-to-all broadcasting operation within the group.

**MPI\_Allgather (&sendbuf,sendcount,sendtype,&recvbuf, recvcount,recvtype,comm)**

## **MPI\_Reduce**

- Applies a reduction operation on all tasks in the group and places the result in one task.

**MPI\_Reduce (&sendbuf,&recvbuf,count,datatype,op,root,comm)**

# Collective Communication Routines

## **MPI\_Allreduce**

- Applies a reduction operation and places the result in all tasks in the group.
- This is equivalent to an MPI\_Reduce followed by an MPI\_Bcast.

**MPI\_Allreduce (&sendbuf,&recvbuf,count,datatype,op,comm)**

## **MPI\_Reduce\_scatter**

- First does an element-wise reduction on a vector across all tasks in the group. Next, the result vector is split into disjoint segments and distributed across the tasks.
- This is equivalent to an MPI\_Reduce followed by an MPI\_Scatter operation.

**MPI\_Reduce\_scatter (&sendbuf,&recvbuf,recvcount,datatype, op,comm)**

# Collective Communication Routines

## **MPI\_Alltoall**

Each task in a group performs a scatter operation, sending a distinct message to all the tasks in the group in order by index.

**MPI\_Alltoall (&sendbuf,sendcount,sendtype,&recvbuf,  
recvcnt,recvtype,comm)**

## **MPI\_Scan**

Performs a scan operation with respect to a reduction operation across a task group.

**MPI\_Scan (&sendbuf,&recvbuf,count,datatype,op,comm)**

# Examples: Collective Communications

```
#include "mpi.h"
#include <stdio.h>
#define SIZE 4
int main(argc,argv)
int argc; char *argv[];
{ int numtasks, rank, sendcount, recvcount, source;
float sendbuf[SIZE][SIZE] = { {1.0, 2.0, 3.0, 4.0},
{5.0, 6.0, 7.0, 8.0}, {9.0, 10.0, 11.0, 12.0},
{13.0, 14.0, 15.0, 16.0} };
float recvbuf[SIZE];
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
```

```
if (numtasks == SIZE)
{ source = 1; sendcount = SIZE; recvcount = SIZE;
MPI_Scatter(sendbuf,sendcount,MPI_FLOAT,recvbuf,recvcount,
MPI_FLOAT,source,MPI_COMM_WORLD);
printf("rank= %d Results: %f %f %f\n",rank,recvbuf[0],
recvbuf[1],recvbuf[2],recvbuf[3]); }
else printf("Must specify %d processors. Terminating.\n",SIZE);
MPI_Finalize();
}
```

# Practice activities

# In-class & take home coding practice

Please try out the following:

- Download and run the following sample MPI code files from this [folder](#)
- In each compiled code, observe the output
- You are also encouraged to attempt the practice activities based on this [document](#).
  - The document contains 5 activities
  - The last activity requires you to parallelise the computation of a pi approximation algorithm. The starter code for these activities are available in the folder linked above.

# Week 6 - Applied Tasks overview

# Week 6 - Applied tasks overview

In Week 6, we will assess our first applied assessments (Applied #1).

- Please read the Week 6 applied specifications and marking guidelines that are available on Moodle (Week 6)
- Discuss with your peers on the tasks in this applied. Refer to your lecture materials and also the sample MPI code discussed in Week 5 to help in the design of the architecture.
- Reach out to your tutor or teaching assistant if you require any clarification on the assessments requirements.